

PLATFORM ARCHITECTURE

A NEXT GENT Build Spec

How to build the foundation once so every app after it takes a week. Data placement, the record, ingestion, capabilities, the app contract, surfaces, the App Store pipeline, isolation, repo layout, and the order to build it in.

01

The one rule

Apps never own data. Apps are views and behaviors over one shared business record.

Get this wrong and you have four SaaS products that need reconciling forever. Get it right and app number seven takes a week.

Everything in this spec exists to enforce that rule mechanically, so it can't be violated by accident at 2am when a feature is due.

02

Where data lives

The question that determines everything else. A tourist scanning a QR code at 9pm on a Saturday cannot depend on a mini PC in a restaurant's back office being reachable — but they also don't need any private data.

Private truth lives on the node. Public projections live on the edge. The node is the only writer.

```

NODE (box on the counter, or a cloud container)
  canonical private state – customers, transactions, credentials,
  visits, bookings, observations, the ledger
  the ONLY thing that writes
    |
    | builds a projection – publishable fields only,
    | signed, versioned, ~kilobytes
    ↓
EDGE (cloud, read-only, CDN-fronted)
  serves: public page · widget · QR target · JSON-LD · MCP endpoint
  holds nothing private, can never write back
    |
    ↓
  tourists · AI agents · partner directories · chambers
  
```

Why this shape and not the alternatives

- **Everything in the cloud** contradicts the entire product thesis and puts you in the breach-liability business.
- **Everything on the box** means a QR menu goes dark when their internet drops, which is a support call per outage and a bad experience for a customer who isn't yours.
- **The split** gives you CDN-speed public surfaces, zero private data in the cloud, and graceful degradation — if the node dies, the edge keeps serving the last projection. Stale, but alive.

PROPERTY The edge is **write-incapable by construction**, not by permission. There is no code path from the edge back into the node. That's what makes "we can't see your data" a structural claim rather than a promise.

What's in a projection

```
projection/v1/{business_id}/{version}.json    (signed)

business    name, hours, location, contact, policies
items       name, category, price, modifiers, dietary,
            rating_avg, rating_count          ← aggregates only
resources    kind, name, available_count      ← counts, never who
reviews      rating, body, item_ref, date      ← no customer identity
availability slots with remaining counts
updated_at, expires_at, signature
```

The exposure rules decide what enters a projection. Nothing else can. A field that isn't in the projection schema physically cannot reach the internet.

03

The node runs anywhere

This is the design decision that makes your free tier possible without a second codebase.

A node is a node whether it sits on their counter or in your cluster.

Same containers, same schema, same capabilities, same everything. Only the location differs.

TIER	NODE LOCATION	INGESTION	EXECUTION
free	your cloud, one container per business	email forward only	none — read and project only
software	their hardware	email + QR + POS	their phone, their browser
ghost	the box you sold them	all sources	all executors, local models, voice

The upgrade is a migration, not a rewrite. A free-tier business that buys a box gets their node shipped to the hardware — same data, same schema, same apps, new location. That path has to exist from day one or the free tier becomes a dead end you'll have to rebuild.

REQUIREMENT Nothing in the node may assume it is on owned hardware. No hardcoded paths, no assumption of a local phone, no assumption of a GPU. Every capability declares what it needs and degrades cleanly when it isn't there.

04

The business record

One schema, shared by every app. Notice which tables multiple apps touch — that overlap is the entire reason the record exists.

```

-- identity and scope
business(id, name, legal_name, vertical, created_at)
location(id, business_id, address, geo, timezone)
member(id, business_id, person_id, role)
node(id, business_id, kind[box|cloud], pubkey, enrolled_at, last_seen)

-- the operating record
resource(id, business_id, kind, name, capacity, attrs)
item(id, business_id, name, category, price_cents, attrs, active)
item_modifier(id, item_id, name, price_delta_cents)
booking(id, business_id, resource_id, channel, external_ref,
        starts_at, ends_at, party_size, status, amount_cents, customer_id)
visit(id, business_id, started_at, ended_at, table_ref, ticket_ref, total_cents)
visit_item(visit_id, item_id, qty, price_cents)
review(id, business_id, visit_id, item_id, rating, body, verified, published_at)

-- private tier, never projected
customer(id, business_id, phone_hash, email_hash, consent, first_seen)
credential_ref(id, business_id, service, vault_ref) -- never the secret

-- provenance: the table you cannot add later
observation(id, business_id, entity_type, entity_id, field, value,
            source, source_ref, observed_at, confidence)
canonical(business_id, entity_type, entity_id, field, value,
          resolved_at, confidence, from_observation_id)

-- action and proof
action(id, business_id, capability, args_hash, requested_by,
       constitution_version, policy_decision, executor,
       started_at, finished_at, status)
evidence(action_id, kind, hash, path, expected, observed)

-- flows
appmap(id, target_app, target_version, capability, version, digest, status)
appmap_run(id, appmap_id, node_id, action_id, result, drift_signature)

-- apps
app(id, package, version, manifest, signature, certified_at)
install(id, business_id, app_id, installed_at, settings)
grant(id, install_id, capability, scope, purpose, expires_at, revoked_at)

```

How canonical works

canonical is a **materialized view**, not a hand-maintained table. A new observation lands, the resolver recomputes that one field, writes the result. Never UPDATE item SET price — always insert an

observation and let resolution decide.

```
resolution policy, per field class:
1  source rank      menu price:  POS > website > Google > Facebook
                        hours:      owner declaration beats everything
2  staleness decay  40 minutes old outranks 6 days old;
                        both decay toward unknown
3  escalation       two high-trust sources disagreeing past a
                        threshold is an owner question, not an
                        automatic resolution
```

CANNOT RETROFIT Provenance is the one thing you cannot add afterward. If you write values directly for six months and then decide you want source tracking, you have six months of data with no origin and no way to reconstruct it. Build the observation table before the first ingestion runs.

One writer per field

FIELD	SOLE WRITER
booking.status	availability app
item.price_cents	menu app (via POS observation)
review.rating	reviews app
visit.*	ingestion, never an app
canonical.*	the resolver, never an app

Two writers on one field is a conflict you will never fully solve. Enforce it in the capability registry, not in a code review.

Ingestion pipeline

SOURCE	→	NORMALIZE	→	OBSERVATION	→	RESOLVE	→	CANONICAL	→	PROJECT
email forward signed biz-a7f3@in... bundle QR scan POS read remote-control read owner text-back partner feed		parser per vendor+format scan event ticket parse screen read declaration iCal / API		insert (never update)		source rank + decay		materialized view		

Email ingestion, concretely

- **Unique inbound address per business** — biz-a7f3@in.yourdomain.com. Never one shared inbox you parse for hints. Routing must be unambiguous from the envelope, and it means you can revoke one business's ingestion without touching anyone else.
- **Parsers are versioned and fingerprinted.** Each parser declares the vendor, the format signature it expects, and what it extracts. A format mismatch means **quarantine and alert**, never a best-effort parse. A wrong booking is worse than a missing one.
- **Raw mail is retained** (encrypted, on the node) until the parse is confirmed, so a fixed parser can reprocess history.
- **Idempotency** on (vendor, external_ref) — the same confirmation forwarded twice must not decrement inventory twice.

ONBOARDING Rank the setup paths by friction: 1 add your address as a second notification email inside the vendor's settings — Toast, Square and FareHarbor all allow it, zero Gmail involvement. 2 a Gmail filter plus a verified forwarding address, two minutes in the UI. 3 an Apps Script, last resort — "run this script" kills conversion at the exact moment you need it to feel like nothing.

Inventory derivation

```

declared capacity (owner)          6 jet skis
- confirmed bookings (all channels, from email)
+ cancellations
- manual decrements (walk-ins, phone)
- safety hold (configurable, covers email lag)
= published availability

reconcile:  read real platform state through the remote control
              on a schedule; compare; flag drift

```

Email lag is 30 seconds to two minutes. Hold a safety slot — show 4 when you have 5 — configurable per business and tightened as you accumulate real lag data.

06

The capability layer

The seam. Apps, agents, the phone assistant, developers and your own code all go through exactly this and nothing else.

A capability is a postcondition plus executors

```

capability: menu.update_price
  args:          { item_id, price_cents }
  postcondition:
    subject:      item{item_id}
    predicate:     price = price_cents
    surfaces:      [website, google, facebook, qr_menu]
    observable_by: [api.read, browser.read, android.read]
  risk:          high
  writes:         [item.price_cents]
  idempotency:    (item_id, price_cents)
  compensation:   roll_back_applied_surfaces
  settle_window:  120s

```

Defining capabilities by **what must become true** rather than by a sequence of steps is what makes executors interchangeable by construction, and it lets verification be derived from the contract instead of written per-executor.

Every call follows the same path

```
caller (app | agent | voice | SMS | developer | MCP)
  ↓ ActionSpec    capability, args, requested_by, purpose
  ↓ constitution  bind the version in force right now
  ↓ policy        ALLOW · ALLOW_WITH_CONSTRAINTS · REQUIRE_APPROVAL · DENY
  ↓ credential    scoped token, never the raw secret
  ↓ router        score executors: login state, health,
                  determinism, reliability, cost, latency, risk
  ↓ execute       api | browser | android | linux | workflow | human
  ↓ verify        read back on a DIFFERENT path than wrote
  ↓ evidence      screenshot, hash, observed vs expected
  ↓ ledger        append-only, immutable
```

Four possible results, not two: VERIFIED, UNVERIFIABLE_YET (a schedulable state, not an error — propagation delay is normal), PARTIAL, CONTRADICTED.

The starting capability set

read	business.get	item.list	review.aggregate_by_item
	resource.availability	booking.list	visit.list
	customer.cohort_count	← count only, never rows	
write	business.update_hours	item.update_price	item.mark_unavailable
	booking.create	booking.cancel	review.publish
	offer.send_to_cohort	← returns a count, never a list	
surface	page.publish	widget.render	qr.generate
	notify.send	projection.build	
system	appmap.run	appmap.report_drift	ledger.append

RULE customers.list does not exist and never will. If a capability's return type is rows of personal data, the whole consent architecture is already lost. Counts, aggregates and actions — never records.

The app contract

Yours and a third-party developer's declare exactly the same thing. That symmetry is what makes it an ecosystem instead of a codebase with plugins.

```
app: restaurant.menu
version: 1.2.0
vertical: restaurant
display: { name: "QR Menu", icon: menu, category: storefront }

reads:
  - item.list
  - review.aggregate_by_item
  - business.get

writes:
  - visit.record_scan

surfaces:
  public_page: { template: menu, blocks: [header, sections, ratings] }
  qr:          { target: public_page, per: table }
  widget:      { sizes: [full, compact] }
  agent_api:   { schema: [Menu, MenuItem, Offer, AggregateRating] }
  voice:       { intents: [whats_on_menu, item_price, is_available] }

settings:
  show_ratings:    { type: bool, default: true }
  currency:        { type: enum, values: [USD], default: USD }
  sections:        { type: list, of: string }
  hide_unavailable: { type: bool, default: true }

requires:
  ingest: [pos]          # optional – degrades without it
  apps:   []

events:
  emits: [menu.scanned, menu.item_viewed]
  listens: [item.price_changed, item.marked_unavailable]

data:
  owns:   []              # apps own nothing
  scopes: [business, item, review, visit]
```

Six rules the manifest enforces

1.

Apps never touch the database. They call capabilities. That seam is what lets you swap storage, add caching, enforce policy and write the ledger without touching app code.
2.

Apps own no tables. `data.owns` is always empty. If an app needs to persist something private to itself, it gets a namespaced key-value scope — never a table in the record.
3.

Surfaces are declared, not built. The app says which surfaces it uses and what blocks to render; the platform renders them. Otherwise you've built seven websites.
4.

Settings are declared, not coded. The platform renders the owner UI from the schema. Otherwise your owner UI becomes seven config screens.
5.

Dependencies are explicit and degrade. `requires.ingest: [pos]` means the app works better with POS and still works without it. Hard dependencies block install; soft ones show a banner.
6.

Events are declared both ways. What it emits and what it listens for, so the platform can wire it without the app knowing who else exists.

08

Surfaces

Build six once. Every app gets all six free, forever.

SURFACE	RUNS ON	WHAT THE APP DECLARES
public_page	edge	template + ordered blocks
widget	edge (iframe / script)	sizes and which blocks
qr	edge target, node-generated	target surface, granularity (per table, per boat, per room)
sms	node, via the SIM	intents it answers, messages it sends
voice	node	intent names and slots
agent_api	edge	JSON-LD types + MCP tool schema

Block rendering

An app declares blocks. The platform owns the templates, the theme, responsiveness, dark mode, SEO and accessibility. An app cannot ship its own CSS or its own page.

```
blocks: [header, hours, sections, ratings, cta_book, reviews, map]
      ↓
platform renders into: public page · widget · JSON-LD · voice response
```

This is why app #7 costs a week. It writes a manifest and some capability calls; it never writes a page, a stylesheet, a settings screen, a QR generator or an SEO tag.

The agent surface is not optional

Every public page emits Schema.org JSON-LD, an MCP endpoint, llms.txt, and /.well-known/agent-card.json — from day one, before anyone asks for it. A restaurant menu as a PDF is invisible to AI. Being the source AI assistants query is an infrastructure position you only get by being there first.

```
public  JSON-LD + llms.txt      free, anonymous, scrapeable
      ↓
identified  MCP / A2A          live availability, item ratings,
                               booking – requires an identified client

→ you get presence from the first, telemetry and the relationship from the second
```

Events

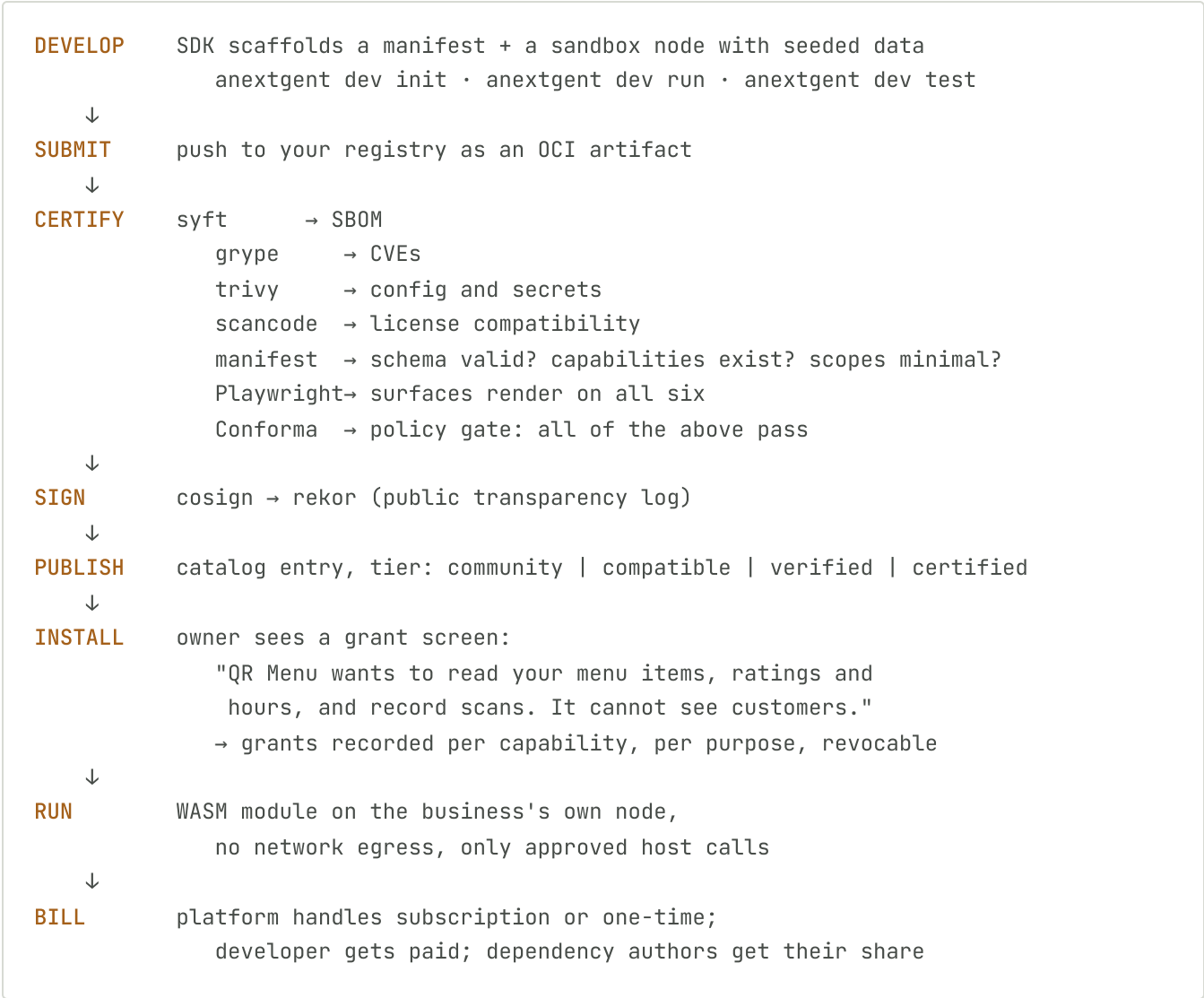
```
ingestion → emits booking.created, item.price_changed, visit.completed
resolver  → emits canonical.changed
apps      → emit  menu.scanned, review.published, offer.sent
executors → emit  action.verified, appmap.drifted
schedule  → emits cron.daily, cron.hourly
```

↓ NATS on the node ↓

```
apps listen      projection rebuild      rulebooks (Event-Driven Ansible)
                                     → re-map drifted flow
                                     → alert the owner
                                     → route to fallback executor
```

DISCIPLINE Events carry **identifiers, not payloads**. `booking.created{booking_id}`, never the booking object. The listener calls a capability to read what it's allowed to read. Payload-carrying events leak data past the policy layer and you'll never find all the places it happened.

The App Store pipeline



Certification tiers

community	Manifest valid, scans pass. No guarantee.
compatible	Automated tests pass on current platform version.
verified	Reviewed and tested by you.
certified	Specific versions, specific hardware, support commitment.

That ladder is a Red Hat innovation and it's how you monetize trust around open source later. Build the field now even if only two tiers are live.

What makes apps compound instead of accumulate

An app that only *consumes* capabilities adds one app. An app that *publishes* one makes every future app easier. The registry knows exactly which app called which — so pay dependency authors from the apps built on them, publish the formula, and publish the payouts. Nobody else can measure this, because nobody else has a registry in the middle.

11

Isolation and tenancy

```
one business  =  one node  =  one database  =  one key

no tenant_id column anywhere in the record
no cross-business query is possible, because there is no
cross-business connection to make
```

Tenant-column multi-tenancy is cheaper to build and leaks eventually — one missing WHERE business_id = and you've had a breach. Node-per-business makes the leak impossible rather than unlikely, and it's what your whole posture depends on.

Inside a node

RUNS AS	ISOLATION	WHY
platform services	Podman + Quadlet	trusted, your code
first-party apps	WASM (Extism / wasmtime)	same rules as everyone else — no privileged path
third-party apps	WASM, no network egress	physically cannot phone home; only approved host functions
untrusted / heavy	gVisor or Firecracker	kernel-level boundary when needed

CRITICAL Your own apps run under the same restrictions as third-party ones. The moment first-party code gets a privileged path to the database, the platform stops being a platform and developers will find out. It also means you discover the SDK's gaps by using it.

Return shaping — the leak nobody plans for

A developer with enough narrow questions reconstructs the dataset. "How many ordered scampi," filtered a thousand ways, rebuilds the customer list one count at a time, and every individual query looks harmless.

- **Minimum cohort size** — results under N (10–25) return nothing
- **Query budget per grant** — not just rate, total questions against a dataset per period
- **Aggregates only for analytics scopes** — action scopes act, they don't report
- **Enumeration detection** in the ledger, flagged to the business

Repo layout

```

anextgent/
  contracts/      ← the owned layer. changes here are versioned events.
    capability/   capability definitions, postconditions, risk
    record/       schema + migrations
    manifest/     app manifest JSON Schema
    actionspec/   the request object
    projection/   what may be published
    constitution/ authority schema

  node/          runs on a box OR in your cloud – identical
    ingest/       email · qr · pos · remote-read · owner-text
      parsers/    per vendor, versioned, fingerprinted
    resolve/      observation → canonical
    registry/     capability registry + router
    policy/       constitution + policy engine + credential broker
    execute/      android · browser · linux · api · workflow · human
    verify/       per-capability postcondition checks
    ledger/       append-only
    project/      builds signed public projections
    host/         WASM app host + host functions
    surface/      sms · voice

  edge/          cloud, read-only, no private data
    page/ widget/ jsonld/ mcp/ qr/

  control/       cloud
    identity/ entitlement/ fleet/ appstore/ billing/ metering/

  apps/          first-party – no privileged access
    menu/ availability/ reviews/ booking/ songrequest/

  sdk/           what a developer installs
  factory/       flow mapping, cert pipeline, parser authoring
  os/            bootc image, quadlets, provisioning

```

DISCIPLINE contracts/ is the only directory that is truly yours forever. Everything else is an implementation of something in it. A change there is a versioned event with a migration path — never an edit.

13

Technology per component

COMPONENT	USE	WHY
node runtime	Podman + Quadlet	systemd units, no daemon, no Kubernetes, identical on box and cloud
node OS	bootc + ostree + greenboot	one signed image, atomic swap, automatic rollback
node database	Postgres	one per node. Same engine cloud and box — don't run SQLite in one and Postgres in the other
embeddings	pgvector	already there
events	NATS	lightweight, runs on a node, no ZooKeeper
app sandbox	Extism + wasmtime	host functions are the only surface; no egress
durable jobs	Temporal	survives crash, reboot, a sleeping phone; resumes exactly
rulebooks	Event-Driven Ansible	sources → rules → actions, already written
android	ADB + uiautomator2 + scrcpy	uiautomator2 is the only runtime executor; Maestro stays in the factory
browser	Playwright	also your verifier for web postconditions
parsing	Tika + Docling + langextract	don't write a parser per file type
inference	llama.cpp + llama-swap, litellm in front	hot-swap, process isolation, one endpoint
guardrail	Granite-Guardian on vLLM	called by the policy engine, no Kubernetes

COMPONENT	USE	WHY
voice	Wyoming + openWakeWord + silero-vad + faster-whisper + Kokoro	each stage swappable; Piper is archived
intents	hassil	deterministic routing before spending a model call
identity	Zitadel	OIDC, passkeys — don't build auth
authorization	SpiceDB	owner → business, agent → VAPP, device → workspace
secrets	OpenBao	behind the credential broker; apps get refs, never secrets
registry	Quay (cloud) + zot (node)	OS images, apps, flow packs — one warehouse
signing	cosign + rekor	and put constitution versions and appmap promotions in the log, not just images
entitlement	Candlepin	what did they buy, what may this node install
fleet	Foreman / flightctl	evaluate flightctl before writing fleet code
edge	Next.js on a CDN	static projections, near-zero cost
owner UI + phone	Flutter	BSD-3, one codebase for wall screen and mobile, no Qt licensing
wall screen	cage (Wayland kiosk)	one app fullscreen, no desktop to fall out of
page builder	Puck	owner-editable public pages over the same blocks
TV nav	Norigin Spatial Navigation	D-pad and remote

COMPONENT	USE	WHY
payments	Stripe Connect	destination charges; decide dispute liability deliberately
billing / metering	Lago	entitlement is "may they," metering is "how much"
networking	headscale / WireGuard	phone app → node, point to point, nothing readable in the middle
backup	restic + Litestream	to a destination they choose; you never hold a key
export	xlsx writer from Postgres	the artifact that makes "you own it" legible

14

Build order

Every milestone produces something demonstrable. Build out of sequence and you rework each step.

Phase A — the record and the seam · weeks 1–5

- 1. contracts/record — schema for **one vertical**, plus observation and canonical
- 2. node/resolve — source rank, staleness decay, escalation
- 3. contracts/capability + node/registry — the first 15 capabilities
- 4. node/ledger — append-only from the very first write
- 5. node/policy — constitution, the four-state ladder, credential broker

DEMO A capability call that resolves policy, executes nothing, and writes a ledger entry. Boring, and everything depends on it.

Phase B — ingestion and projection · weeks 6–10

6. `node/ingest/email` — unique inbound address, parser framework, quarantine, idempotency
7. Three real parsers — one POS, one booking platform, one review notification
8. `node/project` — signed projection bundles
9. `edge/` — page renderer, widget, QR target, JSON-LD, MCP, `llms.txt`, agent card

DEMO A business forwards booking emails and a live public page appears with real availability. That alone is the free tier — no box, no phone, no install.

Phase C — the app host · weeks 11–14

10. `contracts/manifest` + `node/host` — WASM host, host functions, capability-scoped
11. Block rendering — the app declares, the platform renders
12. Settings schema → owner UI generation
13. Install, grant, revoke
14. `sdk/` — init, run, test against a seeded sandbox node

DEMO A hello-world app with fifteen lines of manifest gets a public page, a widget, a QR code, an agent endpoint and a settings screen — none of which it wrote.

Phase D — the first apps · weeks 15–22

In dependency order. Each needs the one before it.

15. **QR Menu** — no ingestion required, immediate value, seeds item
16. **Reviews** — needs visits, which need scans plus POS
17. **Availability** — needs email ingestion. Most valuable, most complex
18. **Booking** — needs availability

Phase E — execution · weeks 23–30

19. `node/execute/android` — adb, uiautomator2, scrcpy
20. AppMap format, fingerprint gate, refuse-to-act

- 21. node/verify — read back on a different path
- 22. Fleet drift aggregation → rulebook → re-map → sandbox → canary → promote
- 23. SMS and voice surfaces

Phase F — the store and the box · weeks 31+

- 24. Certification pipeline, signing, transparency log
- 25. Catalog, tiers, install flow, billing, dependency revenue share
- 26. bootc image, quadlets, factory provisioning, cert rack
- 27. Node migration — cloud tier to owned hardware

15

The first four apps

APP	READS	WRITES	SURFACES	NEEDS
menu	item.list · review.aggregate · _by_item · business.get	visit.record_scan	page · qr · widget · agent · voice	—
reviews	visit.list · item.list	review.publish	sms · page · agent	menu, POS or QR
availability	resource.availabi · lity · booking.list	booking.create · booking.cancel	widget · page · agent · voice	email ingest
booking	resource.availabi · lity · business.get	booking.create · notify.send	page · widget · sms · voice · agent	availability, payments

Look at the overlaps. Menu and reviews both touch item. Availability and booking both touch booking. Reviews needs visits that the menu app creates. If each owned its own copy you'd have four sources of truth for the same fields and permanent reconciliation work — which is exactly what four separate SaaS products are.

Song requests is the proof the platform is real

QR on a stage → page → request → payment confirmation parsed → identity matched → artist notified. **Zero new architecture.** Five capabilities you already built. If it takes a week, the foundation works. If it takes a month, something in Phases A–C is wrong and you should fix that before app number six.

What breaks if you skip

SKIP THIS	WHAT HAPPENS	RECOVERABLE?
observation table	Six months of data with no origin, no confidence, no way to reconstruct. Conflict resolution becomes guesswork forever.	No
one writer per field	Two apps race on booking.status. Intermittent, unreproducible, and it corrupts customer-facing availability.	Painful
capability seam	Apps reach into the database. Policy, ledger and swappable storage all become impossible without rewriting every app.	No
node-per-business	One missing WHERE business_id = is a cross-tenant breach, and your entire posture is gone in one incident.	No
platform-owned surfaces	Seven apps, seven websites, seven settings screens. App #7 costs a month instead of a week.	Expensive
projection boundary	Private fields leak to the edge. The one mistake that ends the company.	No
ledger from write #1	No audit history for the period you most need it, and Proof Ledger becomes a claim you can't back.	No
first-party apps sandboxed	Privileged shortcuts accumulate, the SDK's gaps stay invisible, and developers find out.	Painful
node location-agnostic	Free tier becomes a dead end with no upgrade path, and you rebuild it as a separate product.	Expensive

SKIP THIS	WHAT HAPPENS	RECOVERABLE?
idempotency on ingest	A forwarded confirmation decrements inventory twice. You oversell. The customer loses trust in the number.	Painful

Seven of those ten are **not recoverable** — they'd require rebuilding the foundation with live customers on it. They're all in Phase A and B, they're all cheap to do first, and every one of them is boring.

STILL TRUE

None of Phase E matters if an accessibility service can't run on the apps a Destin charter operator actually uses. One phone, one afternoon, before week 23.

Build spec, revision one. Phase durations assume a small team and will move; the dependency order between phases should not. Companion documents: the Playbook (strategy and economics), the Build Plan (phases and go-to-market), the Parts Catalog (component list with verdicts).